SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 DESIGN **COURSE NAME: COMPILER**

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:100**

Annexure A

CASE STUDY

Code of Conduct:

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan k

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

Annexure A

CASE STUDY

Set 4

1.

A game development company is creating a custom scripting language for controlling character movements and animations. The compiler team uses syntax-directed definitions to generate syntax trees for arithmetic and logical expressions during parsing. Initially, bottom-up evaluation with S-attributed definitions works well for simple expressions, but difficulties arise when inherited attributes are needed for function calls and nested scopes. During semantic analysis, the compiler detects several type mismatches between integer, float, and boolean variables. To ensure smooth execution, the developers implement a type checker with automatic type conversion support.

a) How do syntax-directed definitions assist in syntax tree construction?

b) Why are type conversions necessary in semantic analysis?

2

A banking software firm is designing a secure programming language for financial transactions where semantic correctness is critical. The compiler translates expressions into syntax trees using top-down translation techniques, but complex nested statements cause problems in attribute evaluation. To improve efficiency, the team shifts to bottom-up evaluation of L-attributed definitions for handling variable declarations and parameter passing. During testing, errors occur because equivalent type expressions are not properly verified across modules. The compiler team then integrates a type system capable of checking compatibility and preventing unsafe assignments.

a) Compare top-down and bottom-up evaluation techniques.

b) How does type equivalence improve program reliability?

3

In a compiler research project, students are developing a language that supports arrays, functions, and conditional statements. The parser successfully generates syntax trees, but semantic actions fail when inherited and synthesized attributes are mixed incorrectly. To resolve this issue, the students redesign the syntax-directed definitions using both S-attributed and L-attributed approaches. During semantic checking, the compiler identifies invalid operations involving incompatible type expressions such as arrays and scalar variables. The students therefore implement a simple type checker with explicit type conversion rules.

a) What challenges arise in evaluating inherited and synthesized attributes?

b) Explain the role of type checking in semantic analysis.

4

A software engineering team is developing a compiler for an IoT programming language where commands from sensors must be translated efficiently into intermediate code. The compiler initially uses syntax-directed translation with topdown parsing, but recursive dependencies in semantic rules make translation difficult. To improve semantic processing, the team adopts bottom-up evaluation and constructs syntax trees for all arithmetic and logical expressions. During execution, incorrect results occur because variables with different type expressions are combined without validation. To solve this issue, the compiler introduces type equivalence checking and automatic type conversions.

a) Why are syntax trees important in syntax-directed translation?

b) How do type equivalence and conversion improve compiler accuracy?

5

A university compiler lab is developing a mini language interpreter for mathematical computations. The syntax analyzer generates parse trees successfully, but semantic analysis fails because attribute values are not propagated correctly across grammar productions. To address this issue, the students implement syntax-directed definitions and compare S-attributed and L-attributed evaluation methods. As the language expands to support floating-point and integer operations together, the compiler frequently encounters type mismatch errors. The team then designs a type system capable of checking compatibility and performing safe implicit conversions during translation.

a) Differentiate between S-attributed and L-attributed definitions.

b) Why is a type system necessary in compiler design?

Q1. Game Scripting Language — Syntax-Directed Definitions and Type Conversion

Scenario

A game development company builds a custom scripting language for character movement and animation. Syntax-directed definitions generate syntax trees for arithmetic and logical expressions. Bottom-up S-attributed definitions work for simple expressions, but fail once inherited attributes are needed for function calls and nested scopes. The semantic analyzer also detects type mismatches among integer, float, and boolean variables, prompting the team to add an automatic type-conversion checker.

Key Issue Identified

S-attributed definitions synthesize attributes purely from children and cannot carry contextual information downward. Function calls and nested scopes in the animation scripts genuinely need such downward flow (e.g., an expected parameter type, or the enclosing scope's local variables) — which is exactly where a purely synthesized design breaks, compounded by unchecked mixing of int/float/boolean values.

(a) How do syntax-directed definitions assist in syntax tree construction?

- Each grammar production is paired with a semantic action that both builds a tree node and computes an attribute (value, type, or a pointer to the new node) the instant the production is recognized — e.g., E -> E1 + E2 { E.node = mkNode('+', E1.node, E2.node) }.
- For S-attributed rules this integrates directly into a bottom-up (LR) parser: when a production is reduced, the action runs using the already-computed children on the parser stack, so tree construction happens in the same single pass as parsing — no separate tree-building walk is needed for straightforward expressions such as combining two animation sub-expressions into one binary node.
- The failure case in this scenario — function calls and nested scopes — needs attributes to flow from parent to child (e.g., telling an argument expression what type the parameter expects), which synthesized-only rules structurally cannot express; this is the direct cause of the reported bug.

(b) Why are type conversions necessary in semantic analysis?

- Movement/animation scripts naturally mix float positions and velocities, integer frame counters, and boolean flags (isGrounded, isAttacking); rejecting every mixed-type expression outright would make the language unusably rigid for game designers.
- A defined conversion policy lets the compiler safely widen compatible pairs automatically (int → float in arithmetic, mirroring the resultTypeName widening rule used for a type checker) while still flagging genuinely invalid combinations

(e.g., boolean directly combined with float) as errors — the same principle used to flag int + char* as a type error while allowing int + float.

- Without this distinction the semantic analyzer either over-rejects valid gameplay logic or silently misinterprets values, and either failure mode is worse for a shipped game than a clear compile-time diagnostic.

Proposed Solution / Recommendation

Move to L-attributed definitions (a strict superset of S-attributed) so the missing inherited attributes — expected parameter type, enclosing scope — can be threaded left-to-right through the same single evaluation pass (the same pattern used to pass a running total down through recursive calls). Pair this with a small type-promotion table (int→float allowed; boolean mixed with numeric rejected) attached as a semantic action on every operator production.

Q2. Banking Compiler — Top-Down vs Bottom-Up Evaluation and Type Equivalence

Scenario

A banking software firm designs a secure language for financial transactions, where semantic correctness is critical. Top-down translation of expressions into syntax trees struggles with complex nested statements during attribute evaluation. The team shifts to bottom-up evaluation of L-attributed definitions for variable declarations and parameter passing. Testing reveals errors because equivalent type expressions are not properly verified across modules, so a type system is integrated to check compatibility and prevent unsafe assignments.

Key Issue Identified

Deeply nested financial expressions expose two separate weaknesses at once: a parsing/evaluation strategy (top-down) that re-derives context awkwardly for nested statements, and a type-equivalence gap that lets structurally identical but separately declared types (e.g., two independent "Currency" records) go unrecognized as compatible across modules — both of which are unacceptable in a domain where a wrong assignment has real financial consequences.

(a) Compare top-down and bottom-up evaluation techniques

- Grammar restrictions: top-down (LL) parsing requires a grammar free of left recursion and typically needing left-factoring; bottom-up (LR) parsing handles left recursion naturally and accepts a broader class of grammars.
- Attribute dependency fit: top-down naturally accommodates inherited attributes, since the parent computes and passes context down before descending into each child; bottom-up naturally accommodates synthesized attributes, since a child is fully reduced (and its attributes fully computed) before the parent's action ever fires.
- Parsing efficiency: table-driven LR (bottom-up) parsers are typically more powerful and run in a single deterministic pass over a wider grammar class; hand-written recursive-descent (top-down) parsers are simpler to read and debug but need more grammar massaging.
- Ease of integrating semantic actions: top-down lets actions be placed before, between, and after non-terminal calls — ideal for L-attributed/inherited-attribute-heavy rules such as variable declaration and parameter passing; bottom-up only fires actions at reduction time — ideal for S-attributed rules, and needs the L-attributed "copy rule" restriction to safely support inherited attributes at all.

(b) How does type equivalence improve program reliability?

- Structural type equivalence (comparing base type and structure recursively, not just the declared name) lets the compiler recognize two independently declared but

structurally identical records — such as “Currency” types defined in different modules — as compatible instead of rejecting them purely because of where they were declared.

- This directly prevents unsafe assignments: incompatible amounts (e.g., a raw integer cent count assigned to a structured currency-with-locale type) are caught at compile time, rather than causing silent data corruption mid-transaction.
- A single, uniformly applied equivalence algorithm removes the inconsistency that comes from different teams manually reconciling types module by module, giving predictable, auditable behavior across the whole banking codebase.

Proposed Solution / Recommendation

Adopt bottom-up (LR) parsing with L-attributed definitions restricted to the “copy rule” discipline for variable declarations and parameter passing (as the team already began doing), and implement one shared structural type-equivalence function — comparing base type, field structure, and qualifiers recursively — as the single source of truth used by every module's assignment and parameter checks.

Q3. Mixed Inherited/Synthesized Attributes and Type Checking (Arrays, Functions, Conditionals)

Scenario

A compiler research project builds a language supporting arrays, functions, and conditional statements. The parser generates syntax trees correctly, but semantic actions fail because inherited and synthesized attributes are mixed incorrectly. The team redesigns the syntax-directed definitions using both S-attributed and L-attributed approaches, and later finds invalid operations between incompatible type expressions such as arrays and scalar variables, leading them to implement a type checker with explicit conversion rules.

Key Issue Identified

Arrays and functions need attributes to flow in both directions on the same construct: an array's base type and bounds must flow down into every index expression (inherited), while the index expression's checked, in-range value must flow back up to the enclosing statement (synthesized). Mixing these two flows without a disciplined dependency structure is exactly what breaks evaluation order in the case study.

(a) What challenges arise in evaluating inherited and synthesized attributes together?

- Dependency-graph acyclicity: the attribute dependency graph for a production must be acyclic; freely mixing inherited and synthesized attributes on the same non-terminal can create a back-edge (an inherited attribute depending on a sibling's synthesized attribute that has not been computed yet) unless the grammar obeys the L-attributed restriction (an inherited attribute may depend only on the parent's inherited attributes or on attributes of symbols strictly to its left).
- Parsing-method mismatch: synthesized attributes evaluate naturally bottom-up (during LR reduction); inherited attributes evaluate naturally top-down. Supporting both robustly means either restricting every rule to be L-attributed (single left-to-right pass) or building an explicit dependency graph per production and topologically sorting it before evaluation — real added implementation cost.
- Array/function-specific difficulty: the declaration's base type and dimension bounds must be pushed down into every index or argument expression, while the validated result must be synthesized back up — a genuinely dual flow that a naive “all-synthesized” or “all-inherited” design cannot express, which is precisely where the research team's semantic actions failed.

(b) Explain the role of type checking in semantic analysis

- It detects incompatible operations at compile time — the reported array-vs-scalar mismatch (using an array identifier directly inside scalar arithmetic) is caught by

comparing the operand's structural type (base type plus an “array-of” / dimension constructor) against what the operator expects.

- It prevents unsafe or undefined behaviour — such as treating an array's base address as a plain integer — from ever reaching code generation.
- It gives precise, actionable diagnostics (which two type expressions were compared, and why they conflict), letting developers fix the language definition instead of chasing a confusing runtime failure.

Proposed Solution / Recommendation

Restrict every new array/function grammar rule to be strictly L-attributed, draw out the attribute dependency graph for each production so the team can visually confirm no attribute depends on one not yet computed, and reuse one recursive structural type-equivalence function (comparing base type, array dimensions, and parameter lists) as the single source of truth for every type check in the compiler.

Q4. IoT Compiler — Recursive Dependencies, Bottom-Up Evaluation, and Syntax Trees

Scenario

A software engineering team develops a compiler for an IoT language that translates sensor commands into intermediate code. Top-down parsing with syntax-directed translation runs into recursive dependencies in the semantic rules, making translation difficult. The team adopts bottom-up evaluation and builds syntax trees for all arithmetic and logical expressions. During execution, incorrect results occur because variables with different type expressions are combined without validation, so the team adds type equivalence checking and automatic conversions.

Key Issue Identified

A naive top-down design where an inherited attribute needed early in the descent itself depends on something not yet parsed creates a recursive/cyclic dependency — exactly the "recursive dependencies in semantic rules" problem described. Layered on top, raw sensor integers and calibrated floating-point thresholds are combined with no type validation, risking real-world actuation errors from a purely numeric mistake.

(a) Why are syntax trees important in syntax-directed translation?

- A syntax tree is compact and abstraction-focused: unlike a full parse tree, it drops grammar-bookkeeping nodes (introduced only to remove left recursion or encode precedence) and keeps only operationally meaningful nodes — operators, operands, and control constructs.
- That compactness matters directly for an IoT compiler, which often runs on constrained hardware or under real-time deadlines: a smaller tree means less memory and faster traversal during intermediate-code generation for every sensor command.
- The syntax tree is also the shared representation walked by both semantic analysis and code generation, decoupling "how the source was parsed" from "what it means" — so switching the parsing strategy from top-down to bottom-up, as the team did, required no redesign of the code generator, only of the tree-construction actions.

(b) How do type equivalence and conversion improve compiler accuracy?

- Type equivalence lets the compiler recognize when two sensor-reading types (e.g., structurally identical "Reading" records produced by different driver modules) are genuinely compatible, avoiding both false rejections and, more dangerously, false acceptances of incompatible readings.
- Type conversion rules define exactly which combinations may widen automatically (an integer sensor count safely promoted to float for a threshold comparison) versus which must be explicit — closing the gap that caused the

reported “variables with different type expressions combined without validation” bug.

- Together, they let the compiler catch a bad sensor-command translation — such as treating a raw ADC integer as an already-scaled floating-point voltage — at compile time instead of triggering a faulty actuation in the field.

Proposed Solution / Recommendation

Adopt bottom-up (LR) parsing with S-attributed definitions for command expressions, which removes the recursive-dependency problem outright since every semantic action fires only after its children are fully reduced. Wrap every operator and assignment production with the same structural type-equivalence-and-conversion check used elsewhere in the compiler, applied uniformly to every sensor and control variable.

Q5. University Interpreter — S-Attributed vs L-Attributed and the Need for a Type System

Scenario

A university compiler lab develops a mini language interpreter for mathematical computations. The syntax analyzer generates parse trees successfully, but semantic analysis fails because attribute values are not propagated correctly across grammar productions. The students implement syntax-directed definitions and compare S-attributed and L-attributed evaluation methods; as the language grows to support floating-point and integer operations together, type mismatch errors appear, prompting the team to design a type system for compatibility and safe implicit conversion.

Key Issue Identified

The interpreter's design uses only synthesized flow while some information genuinely needs to travel from parent to child (context such as an expected numeric type or the active variable scope) — exactly what an S-attributed-only design cannot express, which produces the reported "attribute values not propagated correctly" symptom, compounded by the lack of any int/float compatibility rule.

(a) Differentiate between S-attributed and L-attributed definitions

- S-attributed: every attribute is synthesized, computed purely from the attributes of children; there are no inherited attributes at all. This is naturally evaluable bottom-up during LR parsing, with one action firing per reduction — e.g., E.val = E1.val + T.val.
- L-attributed: a strict superset of S-attributed. It permits inherited attributes, but only if each inherited attribute of a right-hand-side symbol depends solely on (i) the parent's inherited attributes, or (ii) attributes of symbols strictly to its left in the same production — never on a symbol to its right.
- That "left-to-right, no forward dependence" restriction is exactly what allows single-pass, left-to-right evaluation — for example, threading a running total down through recursive calls as a function parameter to simulate an inherited attribute.
- Practically, for this interpreter: pure S-attributed rules are simpler and map directly onto the existing bottom-up parser, but cannot pass contextual information (an expected numeric type, or which scope is active) down into subexpressions — precisely the missing capability the case study describes.

(b) Why is a type system necessary in compiler design?

- It formally defines which operations are valid between which type expressions (int, float, and their combinations), replacing ad hoc runtime guessing with a precise, checkable rule set.
- It enables early error detection: a mismatched integer/float operation is caught during semantic analysis instead of silently producing a wrong numeric result mid-

computation — critical for a math interpreter, where a silent precision or type error can invalidate results with no visible symptom.

- It defines a principled, minimal conversion policy (e.g., int promoted to float in mixed arithmetic) so the language stays convenient for students to write in while remaining mathematically sound and predictable.

Proposed Solution / Recommendation

Keep the existing bottom-up parser and S-attributed rules wherever attributes flow purely upward (ordinary arithmetic evaluation), but introduce a small, deliberately limited set of L-attributed inherited attributes (expected type context) specifically for the new mixed int/float expressions, together with a lightweight type system built on one promotion rule (int → float) and one structural equivalence check.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand